



API User's Manual

For the

Newport USB Driver

Prepared by:
Newport Corporation
1791 Deere Avenue
Irvine, CA 92606

1	INTRODUCTION.....	1
2	THE NATIVE INTERFACE.....	1
2.1	INTERFACE FUNCTION GROUPS.....	6
2.1.1	Use USB Address	6
2.1.2	Filter By Product ID	7
2.1.3	Use Device Key	7
2.1.4	Use Event Handling	8
3	THE .NET INTERFACE	9
3.1	LOGGING.....	13
3.2	INTERFACE FUNCTION GROUPS.....	13
3.2.1	Use USB Address	13
3.2.2	Filter By Product ID	15
3.2.3	Use Device Key	15
3.2.4	Use Event Handling	17
4	SAMPLES.....	19
4.1	VB6 SAMPLE	19
4.1.1	UseUSBAddress	19
4.2	C++ SAMPLES	19
4.2.1	UseUSBAddress	19
4.2.2	FilterByProductID	19
4.2.3	UseDeviceKey	20
4.2.4	UseEventHandlingWithDeviceHandle.....	20
4.2.5	UseEventHandlingWithDeviceKey	20
4.3	C# SAMPLES	20
4.3.1	UseUSBAddress	20
4.3.2	FilterByProductID	21
4.3.3	UseDeviceKey	21
4.3.4	UseEventHandling	21
4.4	LABVIEW SAMPLES.....	21
4.4.1	UseUSBAddress	21
4.4.1.1	LabVIEW 6.1	21
4.4.2	FilterByProductID	22
4.4.3	UseDeviceKey	22
4.4.3.1	UseDeviceKey (with Device VIs)	22
4.4.4	UseEventHandling	22

1 Introduction

The Newport USB Driver (UsbDll.dll) allows software running on a PC to communicate with Newport hardware over a Universal Serial Bus. The driver is a Dynamic Link Library made up of custom software from Newport. UsbDll.dll is written in native C++ and the public interface functions are defined and documented in the C++ header file NewpDll.h. Some of these functions have a specific scenario in which they are intended to be used and require special setup before they can be used.

2 The Native Interface

Here are the public interface functions from NewpDll.h:

```
typedef void (__stdcall *DeviceStateChanged)(int handle, int nState);
extern DeviceStateChanged g_lpDeviceStateChangedCB;

// Description:
// This function opens all devices on the USB bus. This function simply calls
// 'newp_usb_open_devices' with 'nProductID' set to zero and with 'bUseUSBAddress' set to true.
// 'newp_usb_open_devices' must be called before any of the other USB functions are called.
//
// Returns:
// Zero for success, non-zero for failure.
//
long EXPORT newp_usb_init_system (void);

// Description:
// This function opens the devices on the USB bus with the specified product ID. This function
// simply calls 'newp_usb_open_devices' with the passed in 'nProductID' and with 'bUseUSBAddress'
// set to true. 'newp_usb_open_devices' must be called before any of the other USB functions are
// called.
//
// Parameters:
// nProductID: The product ID (from NewportUSBDriver.inf), where zero means all products.
//
// Returns:
// Zero for success, non-zero for failure.
//
long EXPORT newp_usb_init_product (int nProductID);

// Description:
// This function opens the devices on the USB bus with the specified product ID and allows the
// USB addressing scheme to be specified. 'newp_usb_open_devices' must be called before any of
// the other USB functions are called.
//
// Parameters:
// nProductID: The product ID (from NewportUSBDriver.inf), where zero means all products.
// bUseUSBAddress: If true then the 'DeviceID' (used in other functions below) is the
// device's USB address, otherwise the 'DeviceID' is the index into the
// device information (which eliminates USB address conflicts). This flag
// should be false if 'DeviceKey' will be used instead of 'DeviceID' to
// reference an open device on the USB bus.
// nNumDevices: Returns the number of devices that were opened.
//
// Returns:
// Zero for success, non-zero for failure.
//
long EXPORT newp_usb_open_devices (int nProductID, bool bUseUSBAddress, int* nNumDevices);

// Description:
// This function closes all devices on the USB bus. After it is called, no USB communication can
// occur until 'newp_usb_open_devices' is called again.
//
void EXPORT newp_usb_uninit_system (void);
```

```

/////////////////////////////////////////////////////////////////
// Description:
// This function initializes event handling for devices with the specified product ID. When a
// device is attached to the USB bus it is opened without affecting the other devices that are
// open or communicating. When a device is detached from the USB bus it is closed without
// affecting the other devices that are open or communicating. This function must be called
// before any of the other USB functions are called.
//
// Parameters:
// nProductID:           The product ID (from NewportUSBDriver.inf), where zero means all
//                        products.
// lpDeviceStateChangedCB: The address of the DeviceStateChanged callback function.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_event_init (int nProductID, DeviceStateChanged lpDeviceStateChangedCB);

/////////////////////////////////////////////////////////////////
// Description:
// This function assigns a unique key to the device referenced by 'handle' and adds it to the
// list of attached devices. This allows an identifier that is more meaningful than the 'handle'
// to be used to reference a specific device.
//
// Parameters:
// DeviceKey:           The device key.
// handle:              The device handle.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_event_assign_key (char* DeviceKey, int handle);

/////////////////////////////////////////////////////////////////
// Description:
// This function removes a device key from the list of attached devices.
//
// Parameters:
// DeviceKey:           The device key.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_event_remove_key (char* DeviceKey);

/////////////////////////////////////////////////////////////////
// Description:
// This function retrieves two arrays of equal size that represent the attached devices:
// one of device keys and another of device handles.
//
// Parameters:
// ppDeviceKeys:        A pointer to an array of null terminated characters,
//                        where each element contains the device key.
// pDeviceHandles:      An integer array of device handles.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_event_get_attached_devices (char** ppDeviceKeys, int* pDeviceHandles);

```

```

/////////////////////////////////////////////////////////////////
// Description:
// This function retrieves the device key that is associated with the specified device handle.
//
// Parameters:
// handle:          The device handle.
// DeviceKey:       The device key.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_event_get_key_from_handle (int handle, char* DeviceKey);

/////////////////////////////////////////////////////////////////
// Description:
// This function retrieves device information for all devices that are open on the USB bus. This
// function must be called in order to determine the proper 'DeviceID' for each open device. The
// data is returned in a character buffer in the following format:
//      <DevID1>,<DevDescription1>;<DevID2>,<DevDescription2>;<DevIDX>,<DevDescriptionX>;
// The data for each device is separated by a semicolon and the data for a device is comma
// delimited. Each 'DeviceID' must be converted to an integer in order to be used with
// 'newp_usb_get_ascii' or 'newp_usb_send_ascii'. The device description data is the same
// response that is returned by a "*"IDN?" query.
//
// Parameters:
// Buffer:          Character buffer used to hold the device information.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_get_device_info (char* Buffer);

/////////////////////////////////////////////////////////////////
// Description:
// This function retrieves an array of device keys used to specify a particular device that is
// open on the USB bus. Each key is unique and consists of the Model and Serial Number strings
// concatenated together (which eliminates USB address conflicts). This function must be called
// before referencing an open device by 'DeviceKey'. Calling this function eliminates the need
// to call 'newp_usb_get_device_info', and allows for the option (if needed) of referencing an
// open device by 'DeviceID' since the array index is the 'DeviceID'.
//
// Parameters:
// ppBuffer:       A pointer to an array of null terminated characters, where the array index
//                 is the 'DeviceID' and each element contains the Model / Serial Number key.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_get_model_serial_keys (char** ppBuffer);

/////////////////////////////////////////////////////////////////
// Description:
// This function retrieves device information for all devices that are open on the USB bus.
//
// arInstruments:   Integer array of device IDs, where each ID is the USB address.
// arInstrumentsModel: Integer array of model numbers.
// arInstrumentsSN:  Integer array of serial numbers.
// nArraySize:      An integer that tells how many elements are in the integer arrays.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT GetInstrumentList (int* arInstruments, int* arInstrumentsModel, int* arInstrumentsSN,
int* nArraySize);

```

```

/////////////////////////////////////////////////////////////////
// Description:
// This function reads the binary response data from the specified device.
//
// Parameters:
// DeviceKey:      The device key.
// Buffer:         Character buffer used to hold the response data.
// Length:        The length of the character buffer.
// BytesRead:     The number of bytes read.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_read_by_key (char* DeviceKey, char* Buffer, unsigned long Length, unsigned
long* BytesRead);

/////////////////////////////////////////////////////////////////
// Description:
// This function reads the binary response data from the specified device.
// It does not expect a NULL terminated character array to be returned, nor does it truncate
// the response data at the carriage-return. The function name was maintained for backwards
// compatibility.
//
// Parameters:
// DeviceID:      The USB address of the device, or the index into the device information.
//                The valid range is from 0 - 31.
// Buffer:         Character buffer used to hold the response data.
// Length:        The length of the character buffer.
// BytesRead:     The number of bytes read.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_get_ascii (long DeviceID, char* Buffer, unsigned long Length, unsigned long*
BytesRead);

/////////////////////////////////////////////////////////////////
// Description:
// This function sends the passed in ASCII command string to the specified device.
// This function concatenates a carriage-return to the command string if it does not have one.
//
// Parameters:
// DeviceKey:      The device key.
// Command:        Character buffer used to hold the command.
// Length:        The length of the character buffer.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_write_by_key (char* DeviceKey, char* Command, unsigned long Length);

/////////////////////////////////////////////////////////////////
// Description:
// This function sends the passed in ASCII command string to the specified device.
// This function concatenates a carriage-return to the command string if it does not have one.
//
// Parameters:
// DeviceID:      The USB address of the device, or the index into the device information.
//                The valid range is from 0 - 31.
// Command:        Character buffer used to hold the command.
// Length:        The length of the character buffer.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_send_ascii (long DeviceID, char* Command, unsigned long Length);

```

```

/////////////////////////////////////////////////////////////////
// Description:
// This function sends the passed in binary data to the specified device.
//
// Parameters:
// DeviceKey:      The device key.
// Command:        Character buffer used to hold the command.
// Length:         The length of the character buffer.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_write_binary_by_key (char* DeviceKey, char* Command, unsigned long Length);

/////////////////////////////////////////////////////////////////
// Description:
// This function sends the passed in binary data to the specified device.
//
// Parameters:
// DeviceID:       The USB address of the device, or the index into the device information.
//                 The valid range is from 0 - 31.
// Command:        Character buffer used to hold the command.
// Length:         The length of the character buffer.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_send_binary (long DeviceID, char* Command, unsigned long Length);

/////////////////////////////////////////////////////////////////
// Description:
// This function reads the ASCII response data from the specified device.
// It expects a NULL terminated character array to be returned with a termination string that
// begins with a carriage-return. The termination string is truncated from the buffer that is
// returned.
//
// Parameters:
// DeviceKey:      The device key.
// Buffer:          Character buffer used to hold the response data.
// Length:         The length of the character buffer.
// BytesRead:      The number of bytes read.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_read_ascii_by_key (char* DeviceKey, unsigned char* Buffer, unsigned long
Length, unsigned long* BytesRead);

/////////////////////////////////////////////////////////////////
// Description:
// This function reads the ASCII response data from the specified device.
// It expects a NULL terminated character array to be returned with a termination string that
// begins with a carriage-return. The termination string is truncated from the buffer that is
// returned.
//
// Parameters:
// DeviceID:       The USB address of the device, or the index into the device information.
//                 The valid range is from 0 - 31.
// Buffer:          Character buffer used to hold the response data.
// Length:         The length of the character buffer.
// BytesRead:      The number of bytes read.
//
// Returns:
// Zero for success, non-zero for failure.
/////////////////////////////////////////////////////////////////
long EXPORT newp_usb_get_ascii_by_DeviceID (long DeviceID, unsigned char* Buffer, unsigned long
Length, unsigned long* BytesRead);

```

```

/////////////////////////////////////////////////////////////////
// Description:
// This function sets the logging flag to the passed in value.
//
// Parameters:
// value:          The boolean value used to set the logging flag.
/////////////////////////////////////////////////////////////////
void newp_usb_SetLogging (bool value);

/////////////////////////////////////////////////////////////////
// Description:
// This function sets the trace logging flag to the passed in value.
//
// Parameters:
// value:          The boolean value used to set the trace logging flag.
/////////////////////////////////////////////////////////////////
void newp_usb_SetTraceLog (bool value);

/////////////////////////////////////////////////////////////////
// Description:
// This function gets the number of discovered devices.
//
// Returns:
// The number of discovered devices.
/////////////////////////////////////////////////////////////////
int newp_usb_GetDeviceCount ();

/////////////////////////////////////////////////////////////////
// Description:
// This function gets the array of device keys (discovered devices).
//
// Parameters:
// ppDeviceKeys:   The array of device keys (discovered devices).
//
// Returns:
// The number of device keys in the list.
/////////////////////////////////////////////////////////////////
int newp_usb_GetDeviceKeys (char** ppDeviceKeys);

/////////////////////////////////////////////////////////////////
// Description:
// This function gets the device key that is associated with the passed in device ID.
//
// Parameters:
// nDeviceID:      The device ID.
// pDeviceKey:     A pointer to the device key being returned.
/////////////////////////////////////////////////////////////////
void newp_usb_GetDeviceKeyFromDeviceID (int nDeviceID, char* pDeviceKey);

/////////////////////////////////////////////////////////////////
// Description:
// This function gets the name of the operating system.
//
// Parameters:
// pOSName:        A pointer to the operating system name being returned.
/////////////////////////////////////////////////////////////////
void newp_usb_GetOSName (char* pOSName);

```

2.1 Interface Function Groups

There are four different ways to initialize and use the USB driver: (1) use USB address, (2) filter by product ID, (3) use device key, and (4) use event handling.

2.1.1 Use USB Address

This case is best used when there is only one instrument attached since the unique identifier used for communication is its USB address. Multiple instruments can be attached, but each one must have a unique USB address assigned before 'newp_usb_init_system' is called, otherwise a

Duplicate USB Address error will be returned. After the instruments on the USB bus have been opened, call 'newp_usb_get_device_info' to get the Device ID of each open instrument. The Device ID is the unique USB address that must be used in each device specific function such as 'newp_usb_get_ascii', 'newp_usb_send_ascii', etc. This means that each instrument is associated with a number from 0 to 31, which is not very descriptive, and it is possible for the USB address of an instrument to be changed by another person or program. Some instruments, such as the power meter, allow the user to change the USB address using its front panel. Others only allow the USB address to be changed programmatically by using a command such as ADDR <address>.

The main functions that belong to this group are:

newp_usb_init_system - this function opens all USB instruments.

newp_usb_get_device_info - this function retrieves the USB address of all open instruments.

newp_usb_get_ascii - this function reads the response data from an instrument.

newp_usb_send_ascii - this function sends the passed in command to an instrument.

newp_usb_get_binary - this function reads the binary response data from an instrument.

newp_usb_send_binary - this function sends the passed in binary data to an instrument.

newp_usb_uninit_system - this function closes all USB instruments.

2.1.2 Filter By Product ID

This case is based upon the previous case "Use USB Address" and so everything is the same except initialization. Instead of calling 'newp_usb_init_system', 'newp_usb_init_product' is called with the Product ID passed in as an argument. If the passed in Product ID is zero, then it behaves just like 'newp_usb_init_system' and opens all USB instruments. If the passed in Product ID is not zero, then only the USB instruments with a matching Product ID are opened. All other USB instruments are not opened and are available for use by other programs.

The Product ID can be obtained from NewportUSBDriver.inf. A portion of this file is listed below that shows the product ID for several instruments. The product ID is a four digit hex number that follows "PID_".

"Newport Power Meter" = USB_Install, USB\VID_104D&PID_CEC7

"New Focus TLB-6800 Tunable Laser Controller" = USB_Install, USB\VID_104D&PID_100D

"New Focus Picomotor Controller" = USB_Install, USB\VID_104D&PID_4000

"Newport ST-300 Smart Table Controller" = USB_Install, USB\VID_104D&PID_2002

"Newport AI-6 Guardian Active Isolation Controller" = USB_Install,
USB\VID_104D&PID_2003

2.1.3 Use Device Key

This case provides an alternative approach, which is more meaningful than the USB address, to referencing a specific instrument. The Device Key is a string that contains the instrument's model name followed by its unique serial number. Since the Device Key is a unique identifier for each instrument, the USB address is not used and can therefore be the same as other instruments. This eliminates the possibility of a Duplicate USB Address error.

To initialize the USB driver for Device Key use, call 'newp_usb_open_devices' with the Product ID parameter set as described in the previous section "Filter By Product ID", and with the Use USB Address parameter set to false. Then call 'newp_usb_get_model_serial_keys' to retrieve the list of Device Keys where the index into the list is the Device ID. After the device keys have been retrieved from the driver, the other methods that accept the Device Key as a parameter can be used ('newp_usb_read_by_key', 'newp_usb_write_by_key', etc.).

The main functions that belong to this group are:

newp_usb_open_devices - this function opens the USB instruments filtered by the Product ID.

newp_usb_get_model_serial_keys - this function retrieves the Device Keys of all open instruments.

newp_usb_read_by_key - this function reads the response data from an instrument.

newp_usb_write_by_key - this function sends the passed in command to an instrument.

newp_usb_read_binary_by_key - this function reads the binary response data from an instrument.

newp_usb_write_binary_by_key - this function sends the passed in binary data to an instrument.

newp_usb_uninit_system - this function closes all USB instruments.

2.1.4 Use Event Handling

This case allows instruments to be dynamically attached (opened) or detached (closed) at any time without affecting the communication that may be occurring among the other attached instruments. This case is different than the others because they open all instruments at the same time and then close them all at the same time. The other cases also do not allow new instruments to be attached when any other instrument is open.

To initialize the USB driver for event handling, call 'newp_usb_event_init' with the Product ID parameter set as described in the earlier section "Filter By Product ID", and with the address of the DeviceStateChanged callback function. After event handling has been initialized, there are two possible ways to handle device I/O.

The simple way is to use the handle of each attached device as the DeviceID on calls made to 'newp_usb_send_ascii' and 'newp_usb_send_ascii'. Using this simple method means that there are no device keys, no list of device information to retrieve, and no need for the other newp_usb_event functions.

The more complex way requires a device key to be created and assigned to a device handle in the USB driver when an instrument is attached. Creating a device key allows an instrument to be identified by a unique name that is more descriptive than a device handle and easier for a user to associate with a particular instrument. When an instrument is detached, the device key that was assigned to a device handle when it was attached must be removed from that list. Once a device key is assigned to a device handle, it can be used in any of the I/O calls listed in the previous section "Use Device Key".

The main functions that belong to this group are:

newp_usb_event_init - this function initializes event handling for instruments with the specified product ID.

newp_usb_event_assign_key - this function assigns a unique key to the instrument referenced by the specified device handle.

newp_usb_event_get_key_from_handle - this function retrieves the device key that is associated with the specified device handle.

newp_usb_event_remove_key - this function removes a device key from the list of attached instruments.

newp_usb_event_get_attached_devices - this function retrieves an array of device keys and another of device handles.

3 The .NET Interface

In addition to the USB driver's native interface that is described above, there is a .NET interface (UsbDllWrap.dll) that can be used by development languages such as C# and LabVIEW. UsbDllWrap.dll uses the native interface functions to communicate with the instrument, but its interface methods wrap around the native functions and provide some enhanced functionality (e.g. logging, device information management, event handling support, and overloaded method calls that allow for different argument types to be passed into the I/O calls).

```

/// <summary>
/// Define the delegate for the Device State Changed event.
/// </summary>
public delegate void DeviceStateChangedDelegate (string deviceKey, eDeviceState state);
/// <summary>
/// The event that is fired when the device state has changed.
/// </summary>
public event DeviceStateChangedDelegate DeviceStateChanged;

/// <summary>
/// Default Constructor.
/// </summary>
public USB ()

/// <summary>
/// Constructor.
/// </summary>
/// <param name="bLogging">True to turn on logging, false to turn off logging.</param>
public USB (bool bLogging)

/// <summary>
/// This property gets the log file path.
/// </summary>
public string LogFilePath

/// <summary>
/// This property gets / sets the logging flag.
/// </summary>
public bool Logging

/// <summary>
/// This function initializes event handling for devices with the specified product ID.
/// When a device is attached to the USB bus it is opened without affecting the other
/// devices that are open or communicating. When a device is detached from the USB bus
/// it is closed without affecting the other devices that are open or communicating.
/// </summary>
/// <param name="productId">The product ID of the devices to open.</param>
/// <returns>True for success, false for failure.</returns>
public bool EventInit (int productId)

```

```

/// <summary>
/// This method parses the passed in identification string for the model and serial number.
/// This method is virtual so that a derived class can provide an alternate implementation.
/// </summary>
/// <param name="ID">The identification string.</param>
/// <param name="deviceKey">The device key.</param>
/// <returns>True for success, false for failure.</returns>
public virtual bool ParseDeviceKey (string ID, ref string deviceKey)

/// <summary>
/// This method gets the attached devices and creates a device table.
/// </summary>
/// <remarks>
/// This method uses the USB driver to build the device table.
/// This class has already built a device table in 'OnDeviceStateChanged'.
/// Therefore it is more efficient to retrieve the device table from 'GetDeviceTable'.
/// </remarks>
/// <returns>A copy of the device table.</returns>
public Hashtable GetAttachedDevices ()

/// <summary>
/// This method opens all USB devices on the bus.
/// </summary>
/// <returns>True for success, false for failure.</returns>
public bool OpenDevices ()

/// <summary>
/// This method opens all USB devices on the bus with the specified product ID.
/// </summary>
/// <param name="productId">The product ID of the devices to open.</param>
/// <returns>True for success, false for failure.</returns>
public bool OpenDevices (int productId)

/// <summary>
/// This method opens all USB devices on the bus with the specified product ID.
/// </summary>
/// <param name="productId">The product ID of the devices to open.</param>
/// <param name="usingDeviceKey">True if referencing devices by key, false if using the USB
address.</param>
/// <returns>True for success, false for failure.</returns>
public bool OpenDevices (int productId, bool usingDeviceKey)

/// <summary>
/// This method closes all USB devices on the bus.
/// </summary>
public void CloseDevices ()

/// <summary>
/// This method reads the response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Read (string deviceKey, StringBuilder buffer)

/// <summary>
/// This method reads the response data from the specified device.
/// </summary>
/// <param name="deviceId">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Read (int deviceId, StringBuilder buffer)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int ReadBinary (string deviceKey, StringBuilder buffer)

```

```

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceId">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int ReadBinary (int deviceId, StringBuilder buffer)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <param name="bytesRead">The number of bytes read from device.</param>
/// <returns>Status code.</returns>
public int ReadBinary (string deviceKey, IntPtr buffer, ref uint bytesRead)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceId">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <param name="bytesRead">The number of bytes read from device.</param>
/// <returns>Status code.</returns>
public int ReadBinary (int deviceId, IntPtr buffer, ref uint bytesRead)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <param name="bytesRead">The number of bytes read from device.</param>
/// <returns>Status code.</returns>
public int ReadBinary (string deviceKey, byte[] buffer, ref uint bytesRead)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceId">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <param name="bytesRead">The number of bytes read from device.</param>
/// <returns>Status code.</returns>
public int ReadBinary (int deviceId, byte[] buffer, ref uint bytesRead)

/// <summary>
/// This method sends the passed in command string to the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <returns>Status code.</returns>
public int Write (string deviceKey, string cmd)

/// <summary>
/// This method sends the passed in command string to the specified device.
/// </summary>
/// <param name="deviceId">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <returns>Status code.</returns>
public int Write (int deviceId, string cmd)

/// <summary>
/// This method sends the passed in binary data to the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The binary data to be sent to the device.</param>
/// <param name="length">The number of bytes data to be sent to the device.</param>
/// <returns>Status code.</returns>
public int WriteBinary (string deviceKey, IntPtr buffer, int length)

```

```

/// <summary>
/// This method sends the passed in binary data to the specified device.
/// </summary>
/// <param name="deviceId">The specified device.</param>
/// <param name="buffer">The binary data to be sent to the device.</param>
/// <param name="length">The number of bytes data to be sent to the device.</param>
/// <returns>Status code.</returns>
public int WriteBinary (int deviceId, IntPtr buffer, int length)

/// <summary>
/// This method sends the passed in binary data to the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The binary data to be sent to the device.</param>
/// <param name="length">The number of bytes data to be sent to the device.</param>
/// <returns>Status code.</returns>
public int WriteBinary (string deviceKey, byte[] buffer, int length)

/// <summary>
/// This method sends the passed in binary data to the specified device.
/// </summary>
/// <param name="deviceId">The specified device.</param>
/// <param name="buffer">The binary data to be sent to the device.</param>
/// <param name="length">The number of bytes data to be sent to the device.</param>
/// <returns>Status code.</returns>
public int WriteBinary (int deviceId, byte[] buffer, int length)

/// <summary>
/// This method sends the passed in command string to the specified device
/// and reads the response data.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Query (string deviceKey, string cmd, StringBuilder buffer)

/// <summary>
/// This method sends the passed in command string to the specified device
/// and reads the response data.
/// </summary>
/// <param name="deviceId">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Query (int deviceId, string cmd, StringBuilder buffer)

/// <summary>
/// This method returns a copy of the device table.
/// </summary>
/// <remarks>
/// The device table is a hash table of devices where the
/// key is the Model / Serial Number and the value is the device ID.
/// </remarks>
/// <returns>A copy of the device table.</returns>
public Hashtable GetDeviceTable ()

/// <summary>
/// This method gets the device keys from the device table.
/// </summary>
/// <param name="deviceKeys">The device keys.</param>
/// <returns>The number of items in the device table.</returns>
public int GetDeviceKeys (out string[] deviceKeys)

/// <summary>
/// This method returns a read only copy of the device information list.
/// </summary>
/// <returns>A read only copy of m_DevInfoList.</returns>
public ArrayList GetDevInfoList ()

```

```

/// <summary>
/// This method returns two arrays that contain the elements of the device information list.
/// </summary>
/// <param name="devIDs">The device IDs of the device information list.</param>
/// <param name="descriptions">The device descriptions of the device information list.</param>
/// <returns>The number of items in the device information list.</returns>
public int GetDevInfoArrays (out int[] devIDs, out string[] descriptions)

/// <summary>
/// This method searches the registry to determine the number of products matching the
/// passed in Vendor / Product key that are currently connected to the USB bus.
/// </summary>
/// <param name="keyVendorProduct">The Vendor / Product key (e.g. "VID_104D&PID_CEC7").</param>
/// <returns>The number of products currently connected to the USB bus.</returns>
public int NumProductsConnected (string keyVendorProduct)

```

3.1 Logging

Logging can be turned on by passing a Boolean true value into the constructor or by setting the Logging property to true. When logging is turned on the following information is written to the log file: general information about the operating system, device initialization data such as the method name, product ID and the device table or device information list, data being written to the device, response data being read from the device, I/O error codes in integer and hex format, event name and state information, and general debugging information. If a C# exception is thrown then this information is logged even if logging is not turned on. Log files are named Log<YYYY-MM-DD>.txt and are created in one of the following folders (depending upon system settings and permissions): (1) the \Log folder located in the directory containing the current executable (.exe), (2) the same folder path as in #1, except the highest level folder is replaced with "My Documents" (e.g. C:\Program Files\Newport\Install Folder\Bin\Log would be replaced with C:\My Documents\Newport\Install Folder\Bin\Log), and (3) the My Documents\Log folder.

3.2 Interface Function Groups

The .NET interface supports the same four interface function groups as the native interface: (1) use USB address, (2) filter by product ID, (3) use device key, and (4) use event handling.

3.2.1 Use USB Address

This case is best used when there is only one instrument attached since the unique identifier used for communication is its USB address. Multiple instruments can be attached, but each one must have a unique USB address assigned before OpenDevices () is called, otherwise false will be returned. After the instruments on the USB bus have been opened, call GetDevInfoList () or GetDevInfoArrays (out int[] devIDs, out string[] descriptions) to get the Device ID of each open instrument. The Device ID is the unique USB address that must be used in each device specific function such as Read (int deviceID, StringBuilder buffer), Write (int deviceID, string cmd), Query (int deviceID, string cmd, StringBuilder buffer), etc. This means that each instrument is associated with a number from 0 to 31, which is not very descriptive, and it is possible for the USB address of an instrument to be changed by another person or program. Some instruments, such as the power meter, allow the user to change the USB address using its front panel. Others only allow the USB address to be changed programmatically by using a command such as ADDR <address>.

The functions that belong to this group are:

```

/// <summary>
/// This method opens all USB devices on the bus.
/// </summary>
/// <returns>True for success, false for failure.</returns>
public bool OpenDevices ()

/// <summary>
/// This method closes all USB devices on the bus.
/// </summary>
public void CloseDevices ()

/// <summary>
/// This method reads the response data from the specified device.
/// </summary>
/// <param name="deviceID">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Read (int deviceID, StringBuilder buffer)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceID">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int ReadBinary (int deviceID, StringBuilder buffer)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceID">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <param name="bytesRead">The number of bytes read from device.</param>
/// <returns>Status code.</returns>
public int ReadBinary (int deviceID, byte[] buffer, ref uint bytesRead)

/// <summary>
/// This method sends the passed in command string to the specified device.
/// </summary>
/// <param name="deviceID">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <returns>Status code.</returns>
public int Write (int deviceID, string cmd)

/// <summary>
/// This method sends the passed in binary data to the specified device.
/// </summary>
/// <param name="deviceID">The specified device.</param>
/// <param name="buffer">The binary data to be sent to the device.</param>
/// <param name="length">The number of bytes data to be sent to the device.</param>
/// <returns>Status code.</returns>
public int WriteBinary (int deviceID, byte[] buffer, int length)

/// <summary>
/// This method sends the passed in command string to the specified device
/// and reads the response data.
/// </summary>
/// <param name="deviceID">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Query (int deviceID, string cmd, StringBuilder buffer)

/// <summary>
/// This method returns a read only copy of the device information list.
/// </summary>
/// <returns>A read only copy of m_DevInfoList.</returns>
public ArrayList GetDevInfoList ()

```



```

/// <summary>
/// This method returns two arrays that contain the elements of the device information list.
/// </summary>
/// <param name="devIDs">The device IDs of the device information list.</param>
/// <param name="descriptions">The device descriptions of the device information list.</param>
/// <returns>The number of items in the device information list.</returns>
public int GetDevInfoArrays (out int[] devIDs, out string[] descriptions)

/// <summary>
/// This method searches the registry to determine the number of products matching the
/// passed in Vendor / Product key that are currently connected to the USB bus.
/// </summary>
/// <param name="keyVendorProduct">The Vendor / Product key (e.g. "VID_104D&PID_CEC7").</param>
/// <returns>The number of products currently connected to the USB bus.</returns>
public int NumProductsConnected (string keyVendorProduct)

```

3.2.2 Filter By Product ID

This case is based upon the previous case “Use USB Address” and so everything is the same except initialization. Instead of calling `OpenDevices ()`, `OpenDevices (int productId)` is called with the Product ID passed in as an argument. If the passed in Product ID is zero, then it behaves just like `OpenDevices ()` and opens all USB instruments. If the passed in Product ID is not zero, then only the USB instruments with a matching Product ID are opened. All other USB instruments are not opened and are available for use by other programs.

The functions that belong to this group are:

```

/// <summary>
/// This method opens all USB devices on the bus with the specified product ID.
/// </summary>
/// <param name="productId">The product ID of the devices to open.</param>
/// <returns>True for success, false for failure.</returns>
public bool OpenDevices (int productId)

```

The Product ID can be obtained from `NewportUSBDriver.inf`. A portion of this file is listed below that shows the product ID for several instruments. The product ID is a four digit hex number that follows “PID_”.

```

"Newport Power Meter" = USB_Install, USB\VID_104D&PID_CEC7
"New Focus TLB-6800 Tunable Laser Controller" = USB_Install, USB\VID_104D&PID_100D
"New Focus Picomotor Controller" = USB_Install, USB\VID_104D&PID_4000
"Newport ST-300 Smart Table Controller" = USB_Install, USB\VID_104D&PID_2002
"Newport AI-6 Guardian Active Isolation Controller" = USB_Install,
USB\VID_104D&PID_2003

```

3.2.3 Use Device Key

This case provides an alternative approach, which is more meaningful than the USB address, to referencing a specific instrument. The Device Key is a string that contains the instrument’s model name followed by its unique serial number. Since the Device Key is a unique identifier for each instrument, the USB address is not used and can therefore be the same as other instruments. This eliminates the possibility of a Duplicate USB Address error.

To initialize the USB driver for Device Key use, call `OpenDevices (int productId, bool usingDeviceKey)` with the Product ID parameter set as described in the previous section “Filter By Product ID”, and with the Using Device Key parameter set to false. Then call `GetDeviceTable ()` to retrieve the table of Device Keys / Device IDs. After the device keys

have been retrieved, the other methods that accept the Device Key as a parameter can be used (Read (string deviceKey, StringBuilder buffer), Write (string deviceKey, string cmd), Query (string deviceKey, string cmd, StringBuilder buffer), etc.).

The main functions that belong to this group are:

```
/// <summary>
/// This method opens all USB devices on the bus with the specified product ID.
/// </summary>
/// <param name="productId">The product ID of the devices to open.</param>
/// <param name="usingDeviceKey">True if referencing devices by key, false if using the USB
address.</param>
/// <returns>True for success, false for failure.</returns>
public bool OpenDevices (int productId, bool usingDeviceKey)

/// <summary>
/// This method closes all USB devices on the bus.
/// </summary>
public void CloseDevices ()

/// <summary>
/// This method reads the response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Read (string deviceKey, StringBuilder buffer)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int ReadBinary (string deviceKey, StringBuilder buffer)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <param name="bytesRead">The number of bytes read from device.</param>
/// <returns>Status code.</returns>
public int ReadBinary (string deviceKey, byte[] buffer, ref uint bytesRead)

/// <summary>
/// This method sends the passed in command string to the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <returns>Status code.</returns>
public int Write (string deviceKey, string cmd)

/// <summary>
/// This method sends the passed in binary data to the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The binary data to be sent to the device.</param>
/// <param name="length">The number of bytes data to be sent to the device.</param>
/// <returns>Status code.</returns>
public int WriteBinary (string deviceKey, byte[] buffer, int length)
```

```

/// <summary>
/// This method sends the passed in command string to the specified device
/// and reads the response data.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Query (string deviceKey, string cmd, StringBuilder buffer)

/// <summary>
/// This method returns a copy of the device table.
/// </summary>
/// <remarks>
/// The device table is a hash table of devices where the
/// key is the Model / Serial Number and the value is the device ID.
/// </remarks>
/// <returns>A copy of the device table.</returns>
public Hashtable GetDeviceTable ()

/// <summary>
/// This method gets the device keys from the device table.
/// </summary>
/// <param name="deviceKeys">The device keys.</param>
/// <returns>The number of items in the device table.</returns>
public int GetDeviceKeys (out string[] deviceKeys)

```

3.2.4 Use Event Handling

This case allows instruments to be dynamically attached (opened) or detached (closed) at any time without affecting the communication that may be occurring among the other attached instruments. This case is different than the others because they open all instruments at the same time and then close them all at the same time. The other cases also do not allow new instruments to be attached when any other instrument is open.

To initialize the USB driver for event handling, subscribe to the DeviceStateChanged event, create a DeviceStateChanged event handler, and call EventInit (int productID) with the Product ID parameter set as described in the earlier section “Filter By Product ID”. After event handling has been initialized, call GetDeviceTable () to retrieve the table of Device Keys / Device IDs. After the device keys have been retrieved, the other methods that accept the Device Key as a parameter can be used (Read (string deviceKey, StringBuilder buffer), Write (string deviceKey, string cmd), Query (string deviceKey, string cmd, StringBuilder buffer), etc.).

The main functions that belong to this group are:

```

/// <summary>
/// Define the delegate for the Device State Changed event.
/// </summary>
public delegate void DeviceStateChangedDelegate (string deviceKey, eDeviceState state);
/// <summary>
/// The event that is fired when the device state has changed.
/// </summary>
public event DeviceStateChangedDelegate DeviceStateChanged;

/// <summary>
/// This function initializes event handling for devices with the specified product ID.
/// When a device is attached to the USB bus it is opened without affecting the other
/// devices that are open or communicating. When a device is detached from the USB bus
/// it is closed without affecting the other devices that are open or communicating.
/// </summary>
/// <param name="productID">The product ID of the devices to open.</param>
/// <returns>True for success, false for failure.</returns>
public bool EventInit (int productID)

```

```

/// <summary>
/// This method reads the response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Read (string deviceKey, StringBuilder buffer)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int ReadBinary (string deviceKey, StringBuilder buffer)

/// <summary>
/// This method reads the binary response data from the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <param name="bytesRead">The number of bytes read from device.</param>
/// <returns>Status code.</returns>
public int ReadBinary (string deviceKey, byte[] buffer, ref uint bytesRead)

/// <summary>
/// This method sends the passed in command string to the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <returns>Status code.</returns>
public int Write (string deviceKey, string cmd)

/// <summary>
/// This method sends the passed in binary data to the specified device.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="buffer">The binary data to be sent to the device.</param>
/// <param name="length">The number of bytes data to be sent to the device.</param>
/// <returns>Status code.</returns>
public int WriteBinary (string deviceKey, byte[] buffer, int length)

/// <summary>
/// This method sends the passed in command string to the specified device
/// and reads the response data.
/// </summary>
/// <param name="deviceKey">The specified device.</param>
/// <param name="cmd">The command string.</param>
/// <param name="buffer">The buffer containing the response data.</param>
/// <returns>Status code.</returns>
public int Query (string deviceKey, string cmd, StringBuilder buffer)

/// <summary>
/// This method returns a copy of the device table.
/// </summary>
/// <remarks>
/// The device table is a hash table of devices where the
/// key is the Model / Serial Number and the value is the device ID.
/// </remarks>
/// <returns>A copy of the device table.</returns>
public Hashtable GetDeviceTable ()

/// <summary>
/// This method gets the device keys from the device table.
/// </summary>
/// <param name="deviceKeys">The device keys.</param>
/// <returns>The number of items in the device table.</returns>
public int GetDeviceKeys (out string[] deviceKeys)

```

4 Samples

The following samples illustrate how to write code that interacts with the Newport USB Driver. The VB6 and C++ samples are written in the native development environment and make use of the USB driver's native interface. The C# and LabVIEW samples make use of the .NET interface to the USB driver.

4.1 VB6 Sample

The VB6 sample code demonstrates how to write code in the native development environment that interacts with the USB driver (UsbDll.dll). The source code can be opened with any text editor. The project files can be opened, built, and debugged with Visual Basic 6.0.

4.1.1 UseUSBAddress

This sample shows how to communicate with an instrument by using the USB address as the unique identifier of each instrument. It opens all devices on the USB bus, retrieves the device information from the USB driver, displays the device information list, selects the first device in the list, performs a write, a read, and then closes all devices.

4.2 C++ Samples

The C++ sample code demonstrates how to write code in the native development environment that interacts with the USB driver (UsbDll.dll). The source code can be opened with any text editor. The solution and project files can be opened, built, and debugged with Visual Studio 2008 or later. Visual Studio 2008 Express, or later (and the Visual Studio Community editions), will also open, build, and debug the entire solution. The Express and Community editions of Visual Studio are free and can be downloaded from Microsoft's web site. It is important to note that the C++ linker requires the input library "usb.dll.lib" to be compatible with the type of target output; otherwise it will produce an error. In other words, if an executable is built to run in 32-bit mode on a 64-bit OS then it must link with the 32-bit "usb.dll.lib" and if an executable is built to run in 64-bit mode then it must link with the 64-bit "usb.dll.lib". When the Newport USB Driver is installed on a 64-bit OS, both the 32-bit and 64-bit driver files are installed. If the 32-bit mode on a 64-bit OS option is installed then the 32-bit files are located in the \Bin folder and the 64-bit files are located in the \Bin\x64 folder. If the 64-bit mode on a 64-bit OS option is installed then the 64-bit files are located in the \Bin folder and the 32-bit files are located in the \Bin\x86 folder.

4.2.1 UseUSBAddress

This sample shows how to communicate with an instrument by using the USB address as the unique identifier of each instrument. It opens all devices on the USB bus, retrieves the device information from the USB driver, displays the device information list, selects the first device in the list, performs a write, a read, and then closes all devices.

4.2.2 FilterByProductID

This sample shows how to communicate with an instrument by using the USB address as the unique identifier of each instrument. It opens all devices on the USB bus with the specified product ID, retrieves the device information from the USB driver, displays the device information list, selects the first device in the list, performs a write, a read, and then closes all devices.

4.2.3 UseDeviceKey

This sample shows how to communicate with an instrument by using the device key as the unique identifier of each instrument. It opens all devices on the USB bus, retrieves the device keys from the USB driver, displays the device table, selects the first device in the table, performs a write, a read, and then closes all devices.

4.2.4 UseEventHandlingWithDeviceHandle

This sample shows how to communicate with an instrument by using event handling with the device handle as the unique identifier of each instrument. First it initializes event handling by indicating that all devices will generate an event and by specifying the address of the event handler callback function. Then when a device is attached or detached, the callback function is invoked by the USB driver. The callback function determines the state of the device. If it is being attached then the device handle is used to write to the device and read its response. If the device is being detached then the device handle is displayed. When the program exits all devices are closed.

4.2.5 UseEventHandlingWithDeviceKey

This sample shows how to communicate with an instrument by using event handling with the device key as the unique identifier of each instrument. First it initializes event handling by indicating that all devices will generate an event and by specifying the address of the event handler callback function. Then when a device is attached or detached, the callback function is invoked by the USB driver. The callback function determines the state of the device. If it is being attached then a device key is created and sent to the USB driver to add to its table of device keys. If the device is being detached then the device key is removed from the USB driver's table of device keys. When the program exits, the first device in the device table is selected, it is written to, read from, the list of attached devices is displayed, and then all devices are closed.

4.3 C# Samples

The C# sample code demonstrates how to write code in the .NET development environment that interacts with the .NET interface (UsbDllWrap.dll) which wraps around the native USB driver (UsbDll.dll). The source code can be opened with any text editor. The solution and project files can be opened, built, and debugged with Visual Studio 2008 or later. Visual Studio 2008 Express, or later (and the Visual Studio Community editions), will also open, build, and debug the entire solution. The Express and Community editions of Visual Studio are free and can be downloaded from Microsoft's web site.

4.3.1 UseUSBAddress

This sample shows how to communicate with an instrument by using the USB address as the unique identifier of each instrument. First it turns on logging by passing in true to the USB class constructor. Then it opens all devices on the USB bus, retrieves the device information, displays the device information list, selects the first device in the list, performs a write, a read, and then closes all devices.

4.3.2 FilterByProductID

This sample shows how to communicate with an instrument by using the USB address as the unique identifier of each instrument. First it turns on logging by passing in true to the USB class constructor. Then it opens all devices on the USB bus with the specified product ID, retrieves the device information from the USB driver, displays the device information list, selects the first device in the list, performs a write, a read, and then closes all devices.

4.3.3 UseDeviceKey

This sample shows how to communicate with an instrument by using the device key as the unique identifier of each instrument. First it turns on logging by passing in true to the USB class constructor. Then it opens all devices on the USB bus, displays the device table, retrieves the device keys from the USB driver, selects a device key, performs a write, a read, and then closes all devices.

4.3.4 UseEventHandling

This sample shows how to communicate with an instrument by using event handling with the device key as the unique identifier of each instrument. First it turns on logging by passing in true to the USB class constructor. Then it initializes event handling by subscribing to the DeviceStateChanged event and indicating that all devices will generate an event. Whenever a device is attached or detached, the event handler is invoked by the USB driver. The event handler determines the state of the device and displays the device key and the new state. When the program exits, it displays the device table, retrieves the device keys from the USB driver, selects a device key, performs a write, a read, and then closes all devices.

4.4 LabVIEW Samples

The LabVIEW sample code demonstrates how to write code in the LabVIEW development environment that interacts with the USB driver in two different ways. The first, and most common, method is to use the .NET interface (UsbDllWrap.dll) which wraps around the native USB driver (UsbDll.dll). The source code can only be opened with LabVIEW 8.0 or later. The project file and its VIs can only be opened, built, and debugged with LabVIEW 8.0 or later. The other way is to interact with the native USB driver (UsbDll.dll) directly. There is only one sample for this and it is located in the UseUsbAddress\LabVIEW 6.1 folder. This sample was developed with LabVIEW 6.1 and can only be opened, built, and debugged with LabVIEW 6.1 or later versions (such as 7.x) that allow it.

4.4.1 UseUSBAddress

This sample shows how to communicate with an instrument by using the USB address as the unique identifier of each instrument. First it turns on logging by passing in true to the USB class constructor. Then it opens all devices on the USB bus, retrieves the device information, displays the device information list, selects the first device in the list, performs a write, a read, and then closes all devices.

4.4.1.1 LabVIEW 6.1

This sample has the same basic functionality as the other samples beneath the UseUSBAddress folder. However, this one is different because it does not use the .NET interface (UsbDllWrap.dll) and the logging that comes with that DLL. Instead it directly interfaces with

the native USB Driver (UsbDll.dll). This sample shows how to communicate with an instrument by using the USB address as the unique identifier of each instrument. It opens all devices on the USB bus, retrieves the device information from the USB driver, displays the device information list, selects the first device in the list, performs a write, a read, and then closes all devices.

4.4.2 FilterByProductID

This sample shows how to communicate with an instrument by using the USB address as the unique identifier of each instrument. First it turns on logging by passing in true to the USB class constructor. Then it opens all devices on the USB bus with the specified product ID, retrieves the device information from the USB driver, displays the device information list, selects the first device in the list, performs a write, a read, and then closes all devices.

4.4.3 UseDeviceKey

This sample shows how to communicate with an instrument by using the device key as the unique identifier of each instrument. First it turns on logging by passing in true to the USB class constructor. Then it opens all devices on the USB bus, displays the device table, selects the first device key, performs a write, a read, and then closes all devices.

4.4.3.1 UseDeviceKey (with Device VIs)

This sample shows how to communicate with an instrument by using the device key as the unique identifier of each instrument and the folder of Device VIs as another way to interact with UsbDllWrap.dll. The Device VIs are simple “wrapper VIs” that usually make a single call to a method in UsbDllWrap.dll. This sample makes the same calls to UsbDllWrap.dll that the UseDeviceKey sample does, but the calls here are made inside of the Device VIs. First it turns on logging by passing in true to the USBInit VI and opens all devices on the USB bus. Then it gets all device keys, selects the first device key, performs a write, a read, and then closes all devices.

4.4.4 UseEventHandling

This sample shows how to communicate with an instrument by using event handling with the device key as the unique identifier of each instrument. First it turns on logging by passing in true to the USB class constructor. Then it calls Event_Register.vi to set up the user defined event “DeviceStateChangedEvent” that is handled in the event case structure. Then it initializes event handling by subscribing to the DeviceStateChanged event and indicating that all devices will generate an event. Whenever a device is attached or detached, the event case is invoked by the USB driver. The event case determines the state of the device and displays the device key and the new state. When the Stop Events button is clicked, the program displays the device table, retrieves the device keys from the USB driver, selects a device key, performs a write, a read, and then closes all devices.